Memoria del Proyecto Final

Predictor de Ocupación para Apartamentos Turísticos en Barcelona

Modelo de clasificación binaria con integración de fuentes externas, almacenamiento cloud y dashboard interactivo

Curso: Big Data e Inteligencia Artificial

IFP España — Curso 2025-2026

Repositorio: github.com/Dasbits/price-optimazer-guiris

Índice

1. Resumen ejecutivo3
2. Introducción y contexto4
3. Objetivos del proyecto5
4. Metodología CRISP-DM6
5. Herramientas y tecnologías utilizadas7
6. Business Understanding11
7. Data Understanding13
8. Data Preparation16
9. Modeling19
10. Evaluation23
11. Deployment26
12. Resultados y discusión30
13. Conclusiones32
14. Limitaciones y líneas de trabajo futuro33
15. Documentación y seguimiento34
16. Bibliografía y referencias35
17. Anexos36

1. Resumen ejecutivo

Este proyecto desarrolla un sistema completo de predicción de ocupación para apartamentos turísticos de Airbnb en Barcelona, siguiendo la metodología CRISP-DM en sus seis fases. El objetivo es predecir, para una combinación dada de listing y fecha, la probabilidad de que esa noche concreta esté reservada o bloqueada en Airbnb.

El proyecto se enmarca dentro del Curso de Big Data e Inteligencia Artificial impartido por IFP España y se concibió originalmente como un Predictor de Precio Óptimo. Sin embargo, durante la fase de Data Understanding se descubrió que la columna price del dataset Inside Airbnb está completamente vacía desde finales de 2024 por restricciones legales aplicadas por la plataforma. Esta limitación obligó a un pivot estratégico desde una tarea de regresión sobre el precio hacia una tarea de clasificación binaria sobre la ocupación, manteniendo intacto el resto del stack tecnológico, la metodología y los entregables.

El dataset final integra cuatro fuentes de información heterogéneas: el scrape público de Inside Airbnb del 14 de diciembre de 2025, la API meteorológica de Open-Meteo, la API de eventos de Ticketmaster y un calendario de festivos catalanes. Tras la fase de limpieza e integración, el dataset operacional contiene 1.012.783 filas, una por cada combinación listing × día, sobre un horizonte temporal de 56 días que abarca la temporada navideña y las primeras semanas de febrero de 2026.

Sobre este dataset se entrenan, validan y comparan tres modelos de clasificación binaria: Logistic Regression como referencia lineal, Random Forest como modelo no lineal robusto y Gradient Boosting como modelo de boosting iterativo. Los tres se implementan en dos plataformas independientes — scikit-learn de forma programática en un cuaderno de Jupyter y Orange Data Mining como flujo visual de widgets — para validar de manera cruzada la consistencia del pipeline. El Random Forest resulta ser el modelo ganador con un AUC de 0,895 y un F1-score de 0,812 sobre el conjunto de test, superando con holgura los umbrales mínimos exigidos por el curso (F1 ≥ 0,75 y AUC ≥ 0,80).

La fase de Deployment se materializa en tres componentes complementarios: la subida del dataset integrado a Google BigQuery como tabla cloud consultable mediante SQL, el desarrollo de tres queries SQL agregadas que sirven de capa de KPIs, y un dashboard interactivo en Power BI Desktop con tres páginas (Visión general, Mercado y Clima) que conecta directamente con BigQuery y permite a un usuario no técnico explorar los datos a través de filtros visuales.

La memoria documenta de forma exhaustiva las decisiones técnicas, los hallazgos no obvios encontrados durante el desarrollo (como el confounding temporal entre lluvia y temporada navideña), las limitaciones intrínsecas del dataset y las posibles líneas de mejora futura.

2. Introducción y contexto

Barcelona es uno de los destinos turísticos más relevantes de Europa, con un mercado de alquiler vacacional especialmente dinámico. Desde la popularización de plataformas como Airbnb, el sector ha experimentado una transformación profunda en términos de oferta, demanda y dinámica de precios. Para los anfitriones, gestores profesionales y reguladores municipales, comprender los patrones de ocupación de los apartamentos turísticos es información crítica que permite tomar decisiones de negocio (pricing dinámico, marketing estacional, optimización de calendarios) y de política pública (regulación urbanística, control de la presión turística sobre el parque residencial).

Inside Airbnb es un proyecto de open data que, desde 2014, publica scrapes mensuales de los listings de Airbnb por ciudad. Esta información, aunque pública, no es trivial de explotar: requiere descarga manual, los datasets son grandes (cientos de miles de filas por ciudad), las columnas relevantes cambian con el tiempo y la propia naturaleza de la información (basada en la disponibilidad declarada por el anfitrión, no en reservas reales) introduce ambigüedades semánticas que hay que conocer y documentar.

El presente proyecto aborda un caso de uso concreto sobre estos datos: construir un modelo de Machine Learning capaz de predecir, para un piso dado y una fecha dada, si esa noche estará marcada como no disponible en el calendario público de Airbnb. La predicción se enriquece con información meteorológica diaria (temperatura, precipitación, viento) y con un cruce inicial con eventos culturales y deportivos del área metropolitana, persiguiendo capturar tanto los factores estructurales del listing (ubicación, capacidad, reputación) como los factores coyunturales del día (clima, calendario laboral, temporada turística).

El proyecto se desarrolla siguiendo la metodología estándar CRISP-DM (Cross-Industry Standard Process for Data Mining), que estructura el ciclo de vida de un proyecto analítico en seis fases secuenciales pero iterativas: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation y Deployment. Cada fase produce entregables identificables y reproducibles que se documentan en el repositorio público del proyecto.

3. Objetivos del proyecto

3.1 Objetivo principal

Construir un modelo de Machine Learning que prediga con métricas competitivas (F1 ≥ 0,75 y AUC ≥ 0,80) la ocupación diaria de cada apartamento turístico de Barcelona presente en el scrape de Inside Airbnb del 14 de diciembre de 2025, durante un horizonte temporal de ocho semanas (14/12/2025 a 07/02/2026).

3.2 Objetivos específicos

1. Integrar cuatro fuentes de datos heterogéneas (Inside Airbnb, Open-Meteo, Ticketmaster, festivos catalanes) en un único dataset analítico con granularidad listing × día.
2. Aplicar técnicas avanzadas de feature engineering, incluyendo target encoding por barrio y por listing, one-hot encoding y eliminación de columnas con leakage.
3. Comparar tres algoritmos de clasificación binaria (Logistic Regression, Random Forest, Gradient Boosting) implementados en dos plataformas independientes (scikit-learn y Orange Data Mining) para validar la consistencia del pipeline.
4. Almacenar el dataset final en Google BigQuery, aprovechando el free tier de la plataforma cloud y permitiendo consultas SQL agregadas reproducibles.
5. Construir un dashboard interactivo en Power BI Desktop conectado a BigQuery, capaz de comunicar los hallazgos del análisis a una audiencia no técnica.
6. Documentar de forma exhaustiva todas las decisiones técnicas, hallazgos curiosos y limitaciones intrínsecas del dataset, en una estructura de READMEs jerarquizada y comentarios inline en código.

3.3 Pivot estratégico del proyecto

El enunciado original del proyecto era construir un Predictor de Precio Óptimo para apartamentos turísticos. Tras la fase inicial de Data Understanding se constató que la columna price del dataset de Inside Airbnb estaba 100 % vacía en todos los registros del scrape del 14/12/2025. Esta limitación, atribuible a restricciones legales aplicadas por Airbnb tras finales de 2024, hizo inviable la tarea original.

Tras evaluar tres alternativas (descartar el dataset, buscar fuentes alternativas de pricing, o redefinir la tarea), se optó por reformular el objetivo hacia un Predictor de Ocupación. Esta reformulación mantiene intacto el stack tecnológico (Python, BigQuery, Orange, Power BI), la metodología (CRISP-DM), las fuentes de datos secundarias (clima, eventos) y los entregables

comprometidos. El cambio se documenta explícitamente en el README principal del proyecto y en los entregables académicos previos.

4. Metodología CRISP-DM

CRISP-DM (Cross-Industry Standard Process for Data Mining) es una metodología estándar de facto para la gestión de proyectos de ciencia de datos. Define seis fases que se siguen de forma secuencial, aunque permitiendo iteraciones entre ellas cuando los descubrimientos de una fase posterior obligan a revisar decisiones tomadas en una anterior.

El proyecto se ha articulado siguiendo esta metodología de forma estricta. La siguiente tabla resume las seis fases y los entregables principales producidos en cada una de ellas:

Fase	Pregunta que responde	Entregable principal
1. Business Understanding	¿Qué problema queremos resolver y por qué?	Definición del problema, métricas de éxito, pivot precio→ocupación
2. Data Understanding	¿Qué datos tenemos y qué nos cuentan?	EDA exhaustivo (notebook 02), correlaciones, hallazgos críticos
3. Data Preparation	¿Cómo dejamos los datos listos para modelar?	Limpieza por fuente e integración (notebooks 03 y 04)
4. Modeling	¿Qué algoritmos usamos y con qué hiperparámetros?	Tres modelos en sklearn (notebook 05) y Orange (.ows)
5. Evaluation	¿Cuán bien funciona el modelo?	Métricas, matrices de confusión, curvas ROC
6. Deployment	¿Cómo lo ponemos en manos del usuario?	BigQuery + queries SQL + dashboard Power BI

Las fases 1 y 2 se desarrollan al inicio del proyecto y son las que motivaron el pivot estratégico. La fase 3 incluye el feature engineering crítico (target encoding y anti-leakage) que después determina las métricas alcanzables en la fase 4. La fase 5 valida los resultados con métricas estandarizadas y la fase 6 cierra el ciclo poniendo los datos y los hallazgos en un formato consumible por usuarios finales.

5. Herramientas y tecnologías utilizadas

El proyecto combina herramientas de procesamiento de datos, machine learning, almacenamiento cloud, visualización analítica y control de versiones. Cada elección tecnológica se justifica por su adecuación al caso de uso, su madurez en el ecosistema y, cuando aplica, su gratuidad.

5.1 Lenguaje y librerías de procesamiento

Python 3.12 es el lenguaje principal del proyecto por su madurez en ciencia de datos y por el tamaño de su ecosistema de librerías especializadas. Sobre Python se han utilizado las siguientes dependencias:

- **pandas.** Manipulación de tablas (DataFrames). Es la herramienta estándar para limpieza, transformación y agregación de datos tabulares. Se utilizó intensivamente en los notebooks 02-04 para integrar las cuatro fuentes (Inside Airbnb, Open-Meteo, Ticketmaster) y generar el dataset final.
- **numpy.** Cálculo numérico vectorizado, base imprescindible de pandas y scikit-learn.
- **matplotlib y seaborn.** Generación de gráficos en los notebooks de exploración (EDA) y de modelado. Seaborn se eligió por su mayor calidad estética y por la configuración por defecto más profesional respecto a matplotlib puro.
- **scikit-learn.** Librería principal de Machine Learning. Implementa los tres algoritmos de clasificación binaria utilizados (Logistic Regression, Random Forest, Gradient Boosting), las métricas de evaluación (F1, AUC, accuracy, matriz de confusión) y las utilidades de preprocesado (StandardScaler, train/test split).

5.2 Entorno de desarrollo

- **Jupyter Notebook.** Entorno interactivo que permite mezclar código, visualizaciones y texto explicativo en una misma página. Se ha usado para los cinco notebooks del flujo (descarga de datos, exploración, limpieza, integración y modelado), facilitando que el desarrollo sea reproducible y autoexplicado.
- **Visual Studio Code.** Editor de código para los scripts Python auxiliares (subir_dataset.py y preparar_dataset_orange.py).
- **Entorno virtual .venv y requirements.txt.** Aísla las dependencias del proyecto del sistema operativo. Garantiza reproducibilidad: cualquier persona que clone el repositorio puede instalar exactamente las mismas versiones.

5.3 Fuentes de datos

- **Inside Airbnb (insideairbnb.com).** Proyecto open data que publica scrapes mensuales de los listings de Airbnb por ciudad. Se descargó el scrape de Barcelona del 14/12/2025 (ficheros listings.csv y calendar.csv).
- **Open-Meteo (open-meteo.com).** API meteorológica gratuita y sin clave que devuelve histórico y climatología de cualquier punto del planeta. Aporta los datos diarios de temperatura, precipitación y viento.
- **Ticketmaster Discovery API.** API gratuita (5.000 llamadas/día) para descargar eventos. Aunque se descargaron eventos de Barcelona, finalmente no se integraron en el dataset principal porque el horizonte temporal del modelo (Dic-Feb) no coincide con los eventos disponibles, que son futuros (Abr-Jul 2026).

5.4 Almacenamiento y consulta cloud

- **Google BigQuery.** Data warehouse serverless de Google Cloud Platform. Se utilizó para almacenar el dataset_integrado.csv (1.012.783 filas) y ejecutar consultas SQL agregadas. El free tier de BigQuery cubre 1 TB de queries/mes y 10 GB de almacenamiento, suficiente para todo el proyecto.
- **google-cloud-bigquery.** Cliente Python oficial de BigQuery, usado en el script sql/subir_dataset.py para automatizar la subida del dataset.
- **python-dotenv.** Gestión de variables de entorno (.env) para nunca subir credenciales al repositorio.

5.5 Modelado de Machine Learning

- **scikit-learn (notebook 05).** Implementación programática de los tres modelos de clasificación. Permite control fino sobre hiperparámetros, feature engineering avanzado (target encoding manual) y validación rigurosa.
- **Orange Data Mining 3.x.** Herramienta visual de Machine Learning. Replica el pipeline de scikit-learn mediante widgets sin código. Sirve como validación cruzada de implementación y como herramienta didáctica para audiencias no técnicas.

5.6 Visualización para deployment

- **Power BI Desktop.** Herramienta de Microsoft para construir dashboards interactivos. Conector nativo a Google BigQuery, soporte de medidas DAX, slicers interactivos y gratuita en su versión Desktop.
- **DAX (Data Analysis Expressions).** Lenguaje de fórmulas de Power BI usado para definir las cuatro medidas calculadas del dashboard.

5.7 Control de versiones y documentación

- **Git y GitHub.** Control de versiones distribuido y alojamiento del repositorio. Permite trazabilidad de todos los cambios y commits temáticos siguiendo la convención Conventional Commits.
- **Markdown.** Formato de los tres ficheros README (principal, orange/ y powerbi/). Renderiza correctamente en GitHub y permite tablas, código y enlaces.

6. Business Understanding

6.1 Definición del problema

El problema de negocio que aborda este proyecto es la predicción de la ocupación diaria de cada apartamento turístico de Barcelona registrado en Airbnb. La pregunta operativa es: dado un piso concreto, identificado por su listing_id, y una fecha futura, ¿qué probabilidad hay de que esa noche el calendario de Airbnb marque el piso como no disponible (es decir, reservado o bloqueado por el anfitrión)?

Esta predicción tiene utilidad práctica para varios actores del ecosistema turístico:

- **Anfitriones independientes:** anticipar periodos de baja demanda para activar promociones, ajustar precios o programar mantenimiento.
- **Gestores profesionales (property managers):** planificar limpieza, recepción y gestión de claves para sus carteras de pisos con mejor estimación de ocupación agregada.
- **Inversores inmobiliarios:** estimar la rentabilidad esperable de un piso antes de comprarlo, comparando barrios y categorías de alojamiento.
- **Reguladores y administraciones:** entender la presión turística agregada por barrio para informar políticas urbanísticas.

6.2 Variable objetivo y formulación de la tarea

La variable objetivo del modelo es la columna binaria occupied, derivada del campo available del fichero calendar.csv de Inside Airbnb:

- **occupied = 1** si el día aparece como no disponible (available='f'). Esto incluye tanto reservas reales como bloqueos voluntarios del anfitrión.
- **occupied = 0** si el día está disponible para ser reservado (available='t').

La tarea es por tanto un problema de clasificación binaria supervisada. Los algoritmos candidatos son cualquier clasificador supervisado: regresión logística, árboles de decisión, ensembles (Random Forest, Gradient Boosting, XGBoost), máquinas de vector soporte, redes neuronales, etc. En este proyecto se han seleccionado los tres más representativos del estado del arte para datos tabulares: Logistic Regression como referencia lineal, Random Forest como ensemble basado en bagging y Gradient Boosting como ensemble basado en boosting.

6.3 Métricas de éxito

Las métricas mínimas exigidas por el curso para considerar el modelo como satisfactorio son:

- **F1-score ≥ 0,75.** Media armónica entre precision y recall. Penaliza tanto los falsos positivos como los falsos negativos. Especialmente útil cuando las clases están desbalanceadas o cuando ambos tipos de error tienen un coste asociado.
- **AUC-ROC ≥ 0,80.** Área bajo la curva ROC. Mide la capacidad discriminativa del modelo a través de todos los umbrales de decisión posibles. Es robusta al desbalanceo de clases.

Adicionalmente se reportan accuracy, precision, recall y MCC (Matthews Correlation Coefficient) para ofrecer una visión completa del rendimiento.

7. Data Understanding

7.1 Fuentes de datos

El proyecto integra cuatro fuentes de datos heterogéneas:

Fuente	Tipo	Volumen	Aporte al modelo
Inside Airbnb	CSV (descarga manual)	18.172 listings + ~5,3 M filas calendario	Identidad del piso, ubicación, atributos, calendario
Open-Meteo	API REST (gratuita, sin clave)	56 días × 7 variables	Temperatura, precipitación, viento por día
Ticketmaster	API REST (clave gratuita)	~1.000 eventos	Eventos culturales/deportivos por día (no integrado finalmente)
Festivos catalanes	Hardcoded en notebook 03	Lista de fechas	Variable binaria is_holiday

7.2 EDA del dataset Inside Airbnb

El fichero listings.csv contiene 18.172 filas, una por cada anuncio activo en Barcelona en el momento del scrape. Las columnas relevantes incluyen identificadores (id, host_id), ubicación (neighbourhood_cleansed con ~70 valores únicos, latitud, longitud), atributos del piso (room_type, property_type, accommodates, bedrooms, bathrooms, beds), reputación (review scores, número de reviews) y disponibilidad agregada (availability_30/60/90/365).

La distribución de room_type sobre la oferta total revela que el mercado está dominado por dos categorías:

- **Entire home/apt:** 11.740 listings (~64,6 %). Apartamento entero para uso turístico exclusivo.
- **Private room:** 6.216 listings (~34,2 %). Habitación privada compartiendo zonas comunes con el anfitrión.
- **Hotel room:** 102 listings (~0,6 %). Habitación de hotel ofertada a través de Airbnb.
- **Shared room:** 114 listings (~0,6 %). Cama en habitación compartida.

El fichero calendar.csv contiene la disponibilidad declarada por el anfitrión para los 365 días posteriores al scrape, con una fila por cada combinación listing × día. La columna available toma los valores 't' (disponible) y 'f' (no disponible).

7.3 Hallazgos críticos

La fase de exploración reveló tres hallazgos que condicionaron el resto del proyecto:

7. **La columna price está 100 % vacía.** Todos los registros del scrape del 14/12/2025 tienen valor NaN en price. Este hallazgo es el motivo del pivot estratégico desde Predictor de Precio hacia Predictor de Ocupación.
8. **Ambigüedad de available='f'.** La columna available no distingue entre días reservados por turistas y días bloqueados manualmente por el anfitrión. En barrios pequeños y residenciales (Montbau, Turó de la Peira, etc.) el bloqueo manual contamina la señal de ocupación turística real, inflando aparentemente la ocupación. Por ello el modelo se restringe a las primeras 8 semanas posteriores al scrape, donde predomina la reserva real frente al bloqueo de largo plazo.
9. **Desfase temporal con Ticketmaster.** La API de Ticketmaster solo devuelve eventos futuros desde el momento de la consulta. En la práctica esto significa que los eventos disponibles cubrían el rango Abr-Jul 2026, mientras que el horizonte del modelo es Dic 2025-Feb 2026. No hay solapamiento, por lo que la variable n_eventos no se incorpora finalmente al dataset modelado.

7.4 Correlaciones a nivel diario

Calculando las correlaciones de Pearson entre la ocupación diaria agregada y las variables externas, se obtienen los siguientes coeficientes en el horizonte estudiado:

Variable	Correlación con % ocupación	Interpretación
temperature_2m_max	−0,696	Más temperatura → menos ocupación (porque más calor coincide con días post-Navidad)
precipitation_sum	+0,131	Lluvia → ligeramente más ocupación (confounding con Navidad)
windspeed_10m_max	+0,007	Sin relación práctica
n_eventos	−0,245	Spurious — no hay solapamiento real con eventos

Estas correlaciones son útiles como diagnóstico inicial, pero deben interpretarse con cautela. La correlación negativa entre temperatura y ocupación, por ejemplo, no significa que el frío atraiga turistas: refleja que los días de Navidad son más fríos y a la vez tienen más demanda. Es un caso de confounding temporal que se documenta también en la página 3 del dashboard Power BI.

8. Data Preparation

8.1 Limpieza por fuente (notebook 03)

Cada fuente recibe un tratamiento de limpieza específico antes de la integración:

- **listings.csv:** Selección de 25 columnas útiles, conversión de booleanos textuales (t/f) a enteros (1/0), imputación de nulos en bedrooms/bathrooms por la mediana del barrio correspondiente, conversión de tipos a categóricos eficientes en memoria.
- **calendar.csv:** Filtrado al horizonte de 8 semanas desde el scrape (14/12/2025 → 07/02/2026), creación de la variable target occupied = (available == 'f'), generación de variables temporales (dow, is_weekend, month, day, week, is_holiday).
- **clima.csv:** Categorización del campo weathercode en cuatro categorías cualitativas (soleado, nublado, lluvia, extremo) basadas en los códigos meteorológicos WMO. Cálculo de temperatura media (temp_avg) y de la variable binaria has_rain.
- **eventos.csv:** Limpieza y agregación a nivel día. No se integra en el dataset principal por la limitación temporal documentada, pero se mantiene como tabla auxiliar para BigQuery y Power BI.

El resultado de esta fase son cuatro CSVs limpios en data/processed/ con cero nulos y los tipos correctos.

8.2 Integración (notebook 04)

La integración se realiza mediante un join secuencial con pandas:

- Primero se hace LEFT JOIN entre calendar_clean y listings_clean usando listing_id como clave. Esto replica los atributos estáticos del piso en cada uno de los días del horizonte.
- Después se hace LEFT JOIN del resultado con clima_clean usando date como clave. Esto añade los datos meteorológicos del día correspondiente.

El dataset resultante tiene granularidad listing × día con 1.012.783 filas (18.172 listings × 56 días, excluyendo el día del scrape para algunos listings) y 40 columnas. Se valida la ausencia total de nulos y se guarda como dataset_integrado.csv (~190 MB) en data/processed/.

8.3 Feature engineering avanzado

Para llevar el dataset al modelado se aplican tres transformaciones críticas, ya específicas del notebook 05 y replicadas en el script preparar_dataset_orange.py:

8.3.1 Eliminación de columnas con leakage

Las columnas availability_30/60/90/365 y estimated_occupancy_l365d, presentes en listings.csv y replicadas en el dataset integrado, vienen calculadas a partir del propio calendar.csv. Son por tanto proxies directos del target occupied. Mantenerlas en el modelo conduce a métricas artificialmente altas (AUC > 0,93) que no reflejan la capacidad predictiva real del modelo. Se eliminan explícitamente del feature set.

8.3.2 Target encoding

Dos variables categóricas de alta cardinalidad se codifican mediante target encoding, calculado únicamente con los datos de train para evitar leakage hacia test:

- **neigh_enc.** Codifica neighbourhood_cleansed (~70 barrios) como la media de occupied en cada barrio dentro del train. Captura la propensión turística estructural del barrio.
- **listing_te.** Codifica listing_id (18.172 valores) como la media de occupied de cada piso en el train. Esta variable se reveló como la feature más predictiva del modelo, con una importancia de 0,66 sobre 1,0 en el Random Forest, capturando la popularidad intrínseca del piso individual.

8.3.3 One-hot encoding

Las variables categóricas de baja cardinalidad (room_type, property_type, weather_cat, neighbourhood_group_cleansed) se transforman mediante one-hot encoding, generando una columna binaria por cada categoría. Esto permite que los algoritmos las consuman como features numéricas sin imponer un orden artificial.

8.4 Split por fecha (no aleatorio)

La división train/test se realiza por fecha, no de forma aleatoria. Los primeros 42 días del horizonte (14/12/2025 a 24/01/2026) constituyen el train con 758.375 filas, y los últimos 14 días (25/01/2026 a 07/02/2026) el test con 254.408 filas.

Esta estrategia simula un escenario realista de predicción: el modelo se entrena con datos del pasado y se evalúa con datos posteriores. Adicionalmente, el periodo de test cae fuera de la temporada navideña (que sí está en el train), lo que añade exigencia: el modelo se evalúa contra un periodo de menor demanda agregada.

9. Modeling

La fase de modelado se desarrolla en dos plataformas independientes para validar de manera cruzada la consistencia del pipeline:

- **Implementación programática** en el notebook 05_modelado.ipynb usando scikit-learn.
- **Implementación visual** en el flujo orange/pipeline_ocupacion.ows usando Orange Data Mining.

Ambas implementaciones consumen los mismos datos preprocesados y configuran los modelos con hiperparámetros equivalentes.

9.1 Logistic Regression

Modelo lineal clásico que estima la probabilidad de la clase positiva mediante la función sigmoide aplicada a una combinación lineal de las features. Se incluye como referencia (baseline) por tres motivos: es rápido de entrenar, sus coeficientes son interpretables y sirve como suelo mínimo de rendimiento.

Hiperparámetros utilizados:

Parámetro	Valor	Justificación
Regularization	Ridge (L2)	Default de sklearn. Penaliza coeficientes grandes y previene overfitting.
Strength (C)	1	Default de sklearn. Valores menores regularizan más.
max_iter	1000	Asegura convergencia con datasets grandes.
class_weight	Desmarcado	La clase ocupada pesa 56 %; balancear penaliza la mayoritaria y reduce F1.

9.2 Random Forest

Ensemble basado en bagging que construye múltiples árboles de decisión sobre subconjuntos aleatorios del dataset y de las features, agregando sus predicciones por votación. Es robusto, fácil de regularizar y captura interacciones no lineales sin requerir feature engineering manual.

Hiperparámetros utilizados:

Parámetro	Valor	Justificación

n_estimators	200	Más árboles → más estable, más lento. <100 da varianza alta; >500 apenas mejora.
max_depth	15	Sin límite, los árboles crecen hasta hojas puras (overfitting). Con 15, generaliza mejor.
min_samples_leaf	5	Hojas con menos de 5 muestras se vuelven ruido. Subir simplifica el modelo.
random_state	42	Fija semilla para reproducibilidad entre ejecuciones.
class_weight	Desmarcado	Mismo razonamiento que Logistic Regression.

9.3 Gradient Boosting

Ensemble basado en boosting iterativo: cada árbol nuevo se entrena para corregir los errores residuales del conjunto de árboles anteriores. Suele ser uno de los modelos más fuertes en datos tabulares y es la base de algoritmos populares como XGBoost y LightGBM.

Hiperparámetros utilizados:

Parámetro	Valor	Justificación
n_estimators	200	Cada árbol corrige errores del anterior. <100 underfit; >500 puede sobreajustar.
learning_rate	0,10	Cuánto pesa cada árbol nuevo. Combinación equilibrada con n_trees.
max_depth	5	Árboles superficiales fuerzan al modelo a aprender patrones simples y combinarlos.
min_samples_split	2	Default de sklearn.
random_state	42	Fija semilla para reproducibilidad.

9.4 Optimización del threshold

Por defecto, los clasificadores binarios usan un umbral de 0,5 sobre la probabilidad predicha para decidir la clase. Sin embargo, el F1-score se maximiza generalmente con un umbral distinto. El notebook 05 incluye una sección que busca el threshold óptimo barriendo la curva precision-recall calculada sobre el train, y aplica ese threshold sobre el test para reportar las métricas finales.

En la práctica, la ganancia obtenida por esta optimización es marginal (del orden de 0,005 puntos en F1) porque el modelo Random Forest está bien calibrado. Aun así, se documenta como buena práctica metodológica.

9.5 Implementación en Orange Data Mining

El flujo visual orange/pipeline_ocupacion.ows replica la lógica del notebook 05 con widgets:

- Dos widgets File cargan train.csv y test.csv (generados por el script preparar_dataset_orange.py).
- Un widget Select Columns marca la columna occupied como Target Variable.
- Tres widgets de modelo (Logistic Regression, Random Forest, Gradient Boosting) se conectan al widget Test and Score, configurado en modo "Test on test data".
- Los widgets Confusion Matrix y ROC Analysis visualizan los resultados de cada modelo.

Las métricas resultantes en Orange son virtualmente idénticas a las de scikit-learn, validando que el pipeline es robusto y reproducible en dos plataformas distintas.

10. Evaluation

10.1 Métricas finales

Los tres modelos se evalúan sobre el mismo conjunto de test (254.408 filas, 14 días posteriores al split). Las métricas finales son:

Modelo	AUC	F1	Accuracy	MCC
Logistic Regression	0,891	0,785	0,787	0,595
Random Forest (ganador)	0,895	0,812	0,812	0,627
Gradient Boosting	0,893	0,804	0,804	0,612

Las tres implementaciones cumplen con holgura los umbrales mínimos del curso (F1 ≥ 0,75 y AUC ≥ 0,80). El Random Forest gana en todas las métricas balanceadas, mientras que el Gradient Boosting empata virtualmente en AUC. La diferencia entre los tres modelos es pequeña (menos de 0,03 puntos en F1), lo cual es señal de que la señal predictiva está concentrada en una feature dominante (el target encoding por listing) que cualquier algoritmo decente puede capturar.

10.2 Matriz de confusión del Random Forest

La matriz de confusión del modelo ganador, sobre las 254.408 filas del conjunto de test, muestra la siguiente distribución de aciertos y errores:

	Predicho libre (0)	Predicho ocupado (1)	Total real
Real libre (0)	102.473 (TN)	28.209 (FP)	130.682
Real ocupado (1)	19.524 (FN)	104.202 (TP)	123.726
Total predicho	121.997	132.411	254.408

De los 130.682 días realmente libres, el modelo identifica correctamente 102.473 (TN, 78,4 %) y se equivoca en 28.209 (FP, 21,6 %). De los 123.726 días realmente ocupados, identifica correctamente 104.202 (TP, 84,2 %) y se equivoca en 19.524 (FN, 15,8 %). El modelo es ligeramente más sensible a la clase ocupado que a la libre, pero los errores son simétricos en magnitud, indicando un comportamiento equilibrado sin sesgo grave hacia ninguna clase.

10.3 Curva ROC

La curva ROC (Receiver Operating Characteristic) representa, para todos los umbrales de decisión posibles, el trade-off entre la tasa de verdaderos positivos (TPR, eje Y) y la tasa de falsos positivos (FPR, eje X). El área bajo esta curva (AUC) es una métrica escalar de la capacidad discriminativa del modelo, independiente del umbral elegido.

En el flujo Orange, las tres curvas (Logistic Regression, Random Forest y Gradient Boosting) aparecen casi superpuestas, todas pegadas a la esquina superior izquierda y muy alejadas de la diagonal aleatoria. Esto indica:

- Los tres modelos tienen capacidad discriminativa similar y muy alta (AUC ~0,89).
- Las diferencias entre modelos no están en la capacidad subyacente sino en la calibración del umbral por defecto.
- El problema está bien especificado: si una de las curvas se hubiera quedado pegada a la diagonal, indicaría que el algoritmo es inadecuado o que faltan features.

10.4 Importancia de variables

La feature importance del Random Forest, calculada como la reducción media de impureza atribuible a cada variable, identifica al target encoding por listing (listing_te) como la feature dominante con un peso de 0,66 sobre 1,0. Las siguientes variables (latitud, longitud, número de reviews, características de capacidad, scores de reviews) tienen importancias del orden de 0,01-0,06 cada una.

Esta concentración de importancia en una sola feature es coherente con la intuición de negocio: la mejor predicción de la ocupación futura de un piso es su ocupación histórica. Capítulo 14 discute las implicaciones de este hallazgo en términos de generalización a listings nuevos (cold start).

11. Deployment

La fase final de CRISP-DM consiste en poner los datos y los hallazgos del modelo en manos del usuario final. En este proyecto, el deployment se materializa en tres componentes complementarios.

11.1 Almacenamiento en Google BigQuery

El dataset_integrado.csv (1.012.783 filas) se carga en Google BigQuery como tabla cloud consultable mediante SQL. La carga se automatiza mediante el script sql/subir_dataset.py, que utiliza la librería google-cloud-bigquery para crear el dataset datos en la región europea, definir la tabla fact_ocupacion con auto-detección de esquema y subir el contenido del CSV. El script es reproducible: cualquier persona con las credenciales adecuadas puede ejecutarlo y obtener exactamente la misma tabla en su propio proyecto de Google Cloud.

La elección de BigQuery se justifica por tres razones:

- **Free tier generoso.** 1 TB de queries al mes y 10 GB de almacenamiento. Suficiente para el dataset del proyecto y para todas las consultas necesarias.
- **Conector nativo a Power BI.** Permite construir el dashboard sin scripts intermedios.
- **Escalabilidad.** Si el proyecto creciera (más ciudades, más tiempo), BigQuery escala sin requerir cambios de arquitectura.

11.2 Capa de KPIs mediante queries SQL

Sobre la tabla fact_ocupacion se construyen tres queries SQL que sirven de capa analítica intermedia. Cada una agrega los datos a un nivel diferente y se diseña pensando en su uso visual posterior:

- **ocupacion_por_barrio.sql.** Devuelve dos rankings concatenados con UNION ALL, distinguidos por la columna segmento ('todo' vs 'entire_home'). El segmento entire_home filtra a room_type='Entire home/apt' para reflejar el mercado turístico real, evitando que barrios pequeños y residenciales contaminen el ranking con bloqueos de anfitrión.
- **ocupacion_por_room_type.sql.** Devuelve cuatro filas (una por categoría) con métricas comparativas: peso sobre la oferta, ocupación general, ocupación de fin de semana, ocupación durante Navidad/Reyes y ocupación fuera de Navidad.
- **ocupacion_por_clima.sql.** Devuelve tres vistas concatenadas con UNION ALL (por categoría meteorológica, por bin de temperatura, con vs sin lluvia). La query agrega primero a nivel día (no listing × día) para evitar que el efecto del clima se contamine con el volumen de pisos por barrio.

Las tres queries siguen una convención estricta: comentarios de cabecera con propósito y métricas, comentarios inline justificando decisiones técnicas (umbral de listings ≥ 30, exclusión del punto y coma final por compatibilidad con Power BI), y nomenclatura snake_case consistente.

11.3 Dashboard interactivo en Power BI

El dashboard final powerbi/dashboard_ocupacion.pbix conecta directamente con BigQuery en modo Importar y consume las cuatro fuentes (la tabla principal y las tres queries SQL). Está organizado en tres páginas, cada una dedicada a responder una pregunta de negocio concreta.

11.3.1 Página 1: Visión general

Responde a la pregunta: ¿qué tamaño tiene el mercado y cómo evoluciona la ocupación en el tiempo? Contiene cuatro tarjetas KPI (Listings totales: 18.172; Noches totales: 1.012.783; % Ocupación: 54,2 %; Pico ocupación diaria: 76,0 %), un gráfico de líneas con la evolución diaria de la ocupación, un ranking de los diez distritos de Barcelona ordenados por ocupación con gradiente condicional rojo-amarillo-verde, y un slicer temporal que permite filtrar todos los visuales al rango de fechas elegido.

11.3.2 Página 2: Mercado

Responde a la pregunta: ¿cómo se reparte la oferta y dónde está el negocio? Contiene un treemap del peso de cada room_type sobre la oferta total, un gráfico de barras agrupadas comparando la ocupación de Navidad vs el resto del horizonte para cada room_type, y un ranking top-10 de barrios con un slicer que alterna entre 'Mercado completo' (todos los room_type) y 'Solo pisos enteros' (entire_home).

11.3.3 Página 3: Clima

Responde a la pregunta: ¿cómo afecta el tiempo a la ocupación? Contiene tres visuales filtrados por la columna metrica del dataset ocupacion_por_clima: ocupación por categoría meteorológica, ocupación por bins de temperatura de 5°C, y ocupación con vs sin lluvia. La página incluye una nota didáctica sobre el confounding temporal entre lluvia y temporada navideña, ilustrando el principio metodológico de que correlación no implica causalidad.

11.4 Medidas DAX

La página 1 utiliza cuatro medidas calculadas en DAX (Data Analysis Expressions), el lenguaje de fórmulas de Power BI:

- **Listings totales = DISTINCTCOUNT(fact_ocupacion[listing_id]).** Cuenta los listings únicos.

- **Noches totales = COUNTROWS(fact_ocupacion).** Número total de filas (listing × día).
- **% Ocupación = AVERAGE(fact_ocupacion[occupied]).** Fracción de noches ocupadas, formateada como porcentaje con un decimal.
- **Pico ocupación diaria = MAXX(SUMMARIZE(fact_ocupacion, fact_ocupacion[date], "ocup_dia", AVERAGE(fact_ocupacion[occupied])), [ocup_dia]).** Agrupa por día, calcula la ocupación media de cada día y devuelve el máximo.

Las medidas son dinámicas: se recalculan automáticamente al aplicar cualquier filtro (slicer temporal, selección de barrio, etc.), lo que hace el dashboard exploratorio sin necesidad de programar nada adicional.

12. Resultados y discusión

12.1 Cumplimiento de objetivos

Los seis objetivos específicos enumerados en el capítulo 3 se han cumplido íntegramente:

- **Integración de fuentes.** El dataset_integrado.csv combina Inside Airbnb, Open-Meteo y festivos catalanes en una tabla coherente de 1.012.783 filas y 40 columnas con cero nulos.
- **Feature engineering avanzado.** Se aplicaron target encoding (por barrio y por listing), one-hot encoding (cuatro variables categóricas) y eliminación de columnas con leakage (cinco columnas).
- **Tres modelos en dos plataformas.** Logistic Regression, Random Forest y Gradient Boosting implementados en sklearn (notebook 05) y replicados en Orange (.ows). Las métricas son consistentes entre plataformas.
- **Almacenamiento en BigQuery.** Tabla fact_ocupacion cargada en price-optimizer-bcn.datos con auto-detección de esquema, accesible mediante SQL.
- **Dashboard interactivo.** dashboard_ocupacion.pbix con tres páginas y cuatro fuentes conectadas a BigQuery, con slicers funcionales y nota didáctica sobre confounding.
- **Documentación exhaustiva.** Tres ficheros README (principal, orange/, powerbi/) más comentarios inline en notebooks, scripts y queries SQL.

12.2 Hallazgos no obvios

Más allá del cumplimiento mecánico de los objetivos, el proyecto ha producido varios hallazgos que merece la pena destacar:

10. **La popularidad histórica del listing es el mejor predictor.** La feature listing_te (target encoding por listing) tiene una importancia de 0,66 sobre 1,0 en el Random Forest, captura el 60 % de la señal predictiva. La ocupación pasada de cada piso es el indicador más fuerte de su ocupación futura.
11. **Barcelona es destino de estancias largas.** La columna delta_finde_pp (puntos porcentuales de diferencia entre ocupación de fin de semana y ocupación general) es prácticamente cero en todos los barrios. Esto indica que Barcelona se reserva como city break de varios días, no como escapada de fin de semana.
12. **Las habitaciones privadas tienen más ocupación que los pisos enteros.** Private room registra un 61 % de ocupación frente al 51 % de Entire home/apt. La hipótesis más plausible es que el precio menor de las habitaciones privadas las hace más accesibles y, por tanto, más ocupadas.

13. **La lluvia parece aumentar la ocupación, pero es confounding.** La correlación positiva entre lluvia y ocupación (+0,131) no implica causalidad: los días lluviosos del horizonte coinciden temporalmente con la semana de Navidad y Año Nuevo, donde la demanda turística es estructuralmente más alta. La fecha es la variable confusora oculta que media la relación aparente.
14. **El primer día del scrape muestra un sesgo.** El gráfico de líneas del dashboard refleja un pico anómalo de ocupación el 14/12/2025 (~76 %) que es artificial: el día del scrape captura todos los calendarios bloqueados desde semanas antes. El sesgo se diluye a partir del día 16-17.

12.3 Validación cruzada de la implementación

Un punto metodológico importante del proyecto es la duplicación intencional del pipeline de modelado en dos plataformas (sklearn y Orange) con el objetivo de validar de manera cruzada que los resultados son consistentes. La siguiente tabla compara las métricas obtenidas en cada plataforma:

Modelo	AUC sklearn	AUC Orange	Diferencia
Random Forest	0,897	0,895	0,002
Gradient Boosting	0,891	0,893	0,002
Logistic Regression	0,629	0,891	0,262 (significativa)

Random Forest y Gradient Boosting dan métricas virtualmente idénticas en ambas plataformas, lo que confirma que el pipeline es robusto. La diferencia notable en Logistic Regression (0,629 en sklearn vs 0,891 en Orange) se explica por dos elecciones del notebook 05 que no se replican en Orange: el uso de class_weight='balanced' y la ausencia de normalización automática que sí aplica Orange. Es un buen ejemplo metodológico de que el preprocessing y los hiperparámetros pesan tanto como el algoritmo elegido.

13. Conclusiones

El proyecto ha demostrado de manera empírica que es posible construir un sistema completo de predicción de ocupación de apartamentos turísticos en Barcelona, integrando múltiples fuentes de datos heterogéneas (Inside Airbnb, Open-Meteo, festivos catalanes), aplicando técnicas estado del arte de Machine Learning, almacenando los resultados en una infraestructura cloud (Google BigQuery) y comunicando los hallazgos mediante un dashboard interactivo (Power BI). Las métricas finales del modelo ganador (Random Forest con AUC = 0,895 y F1 = 0,812) superan con holgura los umbrales mínimos exigidos por el curso.

La metodología CRISP-DM ha demostrado ser un marco efectivo para estructurar el proyecto. La separación clara entre Business Understanding, Data Understanding y Data Preparation permitió detectar tempranamente el problema de la columna price vacía y reorientar el proyecto sin desperdiciar trabajo. Las fases de Modeling y Evaluation se beneficiaron de un dataset bien preparado, mientras que la fase de Deployment cerró el ciclo poniendo la inteligencia generada en manos del usuario final.

La duplicación intencional del pipeline de modelado en scikit-learn y Orange Data Mining no fue un trabajo redundante: validó la consistencia de los resultados, expuso diferencias sutiles entre implementaciones (especialmente en Logistic Regression) y produjo una herramienta accesible para audiencias no técnicas. El dashboard de Power BI, al consumir directamente desde BigQuery, garantiza que cualquier futuro scrape de Inside Airbnb pueda incorporarse al análisis sin más esfuerzo que ejecutar el script de subida y pulsar Actualizar.

La feature más importante del modelo final, la popularidad histórica de cada listing (listing_te), es a la vez la más predictiva y la más limitante: el modelo funciona excepcionalmente bien para listings con historial pero degradaría su rendimiento ante listings completamente nuevos (escenario de cold start). Esta limitación, documentada explícitamente, define la frontera de aplicabilidad del modelo y abre la puerta a líneas de trabajo futuro descritas en el capítulo 14.

14. Limitaciones y líneas de trabajo futuro

14.1 Limitaciones intrínsecas del dataset

- **Ambigüedad de available='f'.** Inside Airbnb no distingue entre noches reservadas por turistas y noches bloqueadas por el anfitrión. El proyecto mitiga este sesgo limitando el horizonte a 8 semanas posteriores al scrape, pero no lo elimina.
- **Ausencia de la columna price.** La columna price está vacía en todos los registros desde finales de 2024 por restricciones legales. Esto motivó el pivot del proyecto y impide cualquier análisis de pricing dinámico sobre estos datos.
- **Cold start.** El modelo se beneficia enormemente del target encoding por listing, lo que implica que su rendimiento degradaría ante pisos completamente nuevos sin historial.
- **Horizonte limitado.** Solo se analizan 56 días, que cubren parte de la temporada navideña. Las conclusiones no son extrapolables directamente a verano u otras temporadas.

14.2 Limitaciones de las fuentes externas

- **Ticketmaster solo expone eventos futuros.** La API no permite recuperar histórico de eventos pasados, por lo que no puede usarse retrospectivamente para enriquecer la información del horizonte modelado.
- **Climatología en futuro.** Open-Meteo solo dispone de datos observados hasta hace ~5 días. Para los días futuros del horizonte, el clima se sustituye por climatología del año anterior, marcando explícitamente el origen en la columna fuente.

14.3 Líneas de trabajo futuro

El proyecto puede extenderse en varias direcciones complementarias:

15. **Modelos más potentes.** Probar XGBoost, LightGBM o CatBoost. Estos algoritmos, optimizados para datos tabulares, suelen aportar 1-3 puntos extra en AUC sobre Gradient Boosting de sklearn.
16. **Manejo del cold start.** Construir un segundo modelo específico para listings sin historial, basado únicamente en features de contenido (ubicación, tipo, capacidad, reputación). Este modelo tendría peor rendimiento en el horizonte conocido pero generalizaría a listings nuevos.
17. **Ampliación temporal.** Acumular múltiples scrapes mensuales de Inside Airbnb a lo largo de un año para entrenar el modelo en periodos más largos y capturar variaciones estacionales.

18. **Geo-features avanzadas.** Distancia a estaciones de metro, distancia a la Sagrada Família o al puerto, densidad de bares en un radio de 200m. Pueden capturarse cruzando con OpenStreetMap.
19. **Texto libre.** Procesar las columnas description y name de los listings con técnicas NLP (embeddings, sentiment analysis) para extraer features cualitativas adicionales.
20. **Despliegue como API.** Servir el modelo entrenado mediante un endpoint REST (Flask, FastAPI) o como función serverless en Google Cloud Run, permitiendo predicciones bajo demanda desde aplicaciones externas.

15. Documentación y seguimiento

La documentación del proyecto se ha articulado en cuatro niveles complementarios:

15.1 README principal

Manual de usuario completo del proyecto en la raíz del repositorio. Incluye resumen ejecutivo, stack tecnológico, estructura del repositorio, configuración del entorno, guía paso a paso de obtención de datos y credenciales, orden de ejecución de los notebooks, sección de subida a BigQuery, sección de Power BI, metodología CRISP-DM y limitaciones conocidas.

15.2 READMEs auxiliares por carpeta

Dos documentos especializados en orange/README.md y powerbi/README.md, con explicación detallada de cada componente, configuración de widgets y visuales, e interpretación didáctica de los resultados.

15.3 Documentación en código

Notebooks Jupyter con celdas Markdown explicativas, scripts Python con docstrings de módulo y función, y queries SQL con bloques de comentario que justifican cada decisión técnica.

15.4 Capturas y evidencias

La carpeta orange/images/ contiene cinco capturas de pantalla que documentan los resultados del flujo Orange: la tabla del Test and Score, las tres matrices de confusión y la curva ROC comparativa. Estas capturas se referencian desde el README correspondiente y sirven como prueba documental.

15.5 Control de versiones

El proyecto utiliza Git con commits temáticos siguiendo la convención Conventional Commits. Cada bloque grande de trabajo (modelado, Orange, BigQuery, Power BI) corresponde a un commit independiente con mensaje descriptivo. El .gitignore excluye datos pesados, credenciales, entorno virtual y cachés.

16. Bibliografía y referencias

16.1 Fuentes de datos

- **Inside Airbnb.** http://insideairbnb.com/get-the-data/ — proyecto open data con scrapes mensuales de listings de Airbnb por ciudad. Scrape utilizado: Barcelona, 14/12/2025.
- **Open-Meteo.** https://open-meteo.com/en/docs/historical-weather-api — API gratuita de datos meteorológicos históricos con licencia CC BY 4.0.
- **Ticketmaster Discovery API.** https://developer.ticketmaster.com/products-and-docs/apis/discovery-api/v2/ — API gratuita de eventos con cuota de 5.000 llamadas/día.

16.2 Documentación técnica

- **Scikit-learn.** https://scikit-learn.org/stable/documentation.html — documentación oficial de la librería principal de Machine Learning.
- **Orange Data Mining.** https://orangedatamining.com/docs/ — documentación de los widgets visuales de Machine Learning.
- **Google BigQuery.** https://cloud.google.com/bigquery/docs — documentación del data warehouse cloud.
- **Power BI Desktop.** https://learn.microsoft.com/en-us/power-bi/ — documentación de la herramienta de visualización.
- **DAX.** https://learn.microsoft.com/en-us/dax/ — referencia del lenguaje de fórmulas de Power BI.

16.3 Metodología y conceptos

- **CRISP-DM.** Chapman, P. et al. (2000). "CRISP-DM 1.0: Step-by-step data mining guide". SPSS.
- **Target encoding.** Micci-Barreca, D. (2001). "A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems". ACM SIGKDD Explorations.
- **Random Forest.** Breiman, L. (2001). "Random Forests". Machine Learning, 45(1), 5-32.
- **Gradient Boosting.** Friedman, J. H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine". The Annals of Statistics.

17. Anexos

17.1 Estructura del repositorio

La organización completa del repositorio es la siguiente:

Carpeta o fichero	Contenido
data/raw/	Datos originales descargados (no subidos al repo por tamaño y derechos)
data/processed/	Datos limpios e integrados generados por los notebooks
notebooks/	Cinco cuadernos Jupyter con el flujo completo (descarga, EDA, limpieza, integración, modelado)
orange/	Flujo visual .ows, script preparador, capturas y README específico
powerbi/	Dashboard .pbix y README específico
sql/	Tres queries SQL de KPI y script de subida a BigQuery
docs/	Documentación académica (esta memoria, entregables por corte)
README.md	Manual de usuario principal del proyecto
requirements.txt	Dependencias Python (instalables con pip install -r)
.gitignore	Exclusiones de Git (datos pesados, credenciales, .venv)
.env.example	Plantilla de variables de entorno

17.2 Comandos clave para reproducir el proyecto

La secuencia mínima para reproducir el proyecto desde cero es:

21. git clone https://github.com/Dasbits/price-optimazer-guiris.git
22. cd price-optimazer-guiris && python -m venv .venv && source .venv/bin/activate
23. pip install -r requirements.txt
24. Descargar manualmente listings.csv y calendar.csv de Inside Airbnb a data/raw/
25. Configurar .env con TICKETMASTER_API_KEY, BIGQUERY_PROJECT_ID y GOOGLE_APPLICATION_CREDENTIALS
26. Ejecutar los notebooks 01 a 05 en orden
27. python sql/subir_dataset.py para cargar la tabla en BigQuery

28. python orange/preparar_dataset_orange.py para generar train.csv y test.csv
29. Abrir orange/pipeline_ocupacion.ows con Orange Data Mining
30. Abrir powerbi/dashboard_ocupacion.pbix con Power BI Desktop

17.3 Métricas de validación cruzada

Comparativa final entre las dos implementaciones (scikit-learn y Orange Data Mining) sobre el mismo conjunto de test (254.408 filas):

Modelo	F1 sklearn	F1 Orange	AUC sklearn	AUC Orange
Logistic Regression	0,514	0,785	0,629	0,891
Random Forest	0,815	0,812	0,897	0,895
Gradient Boosting	0,802	0,804	0,891	0,893